



FTP & CHAT SYSTEM REPORT

CA1 - Computer Networks



APRIL 8, 2026

MAHDY MOKHTARI - 810101515

<https://github.com/MahdyMokh7/computerNetworking-miniProjects/tree/master/CA1-FTPChatProject>
- 36d733829138af801918d53ac4797a628a531f8c

1. Introduction

This project implements a simplified version of the File Transfer Protocol (FTP) combined with a group chat system using TCP socket programming in C++. The system follows a client-server architecture where multiple clients can connect simultaneously, exchange messages publicly or privately, and transfer files to/from the server.

Key features implemented:

- Real-time group messaging
- Private messaging between users
- Online user listings
- File upload/download with progress bars
- Concurrent multi-client handling

The communication occurs over a single **TCP connection** using a custom pipe-delimited protocol for command and data exchange.

2. System Architecture

2.1 Overall Design

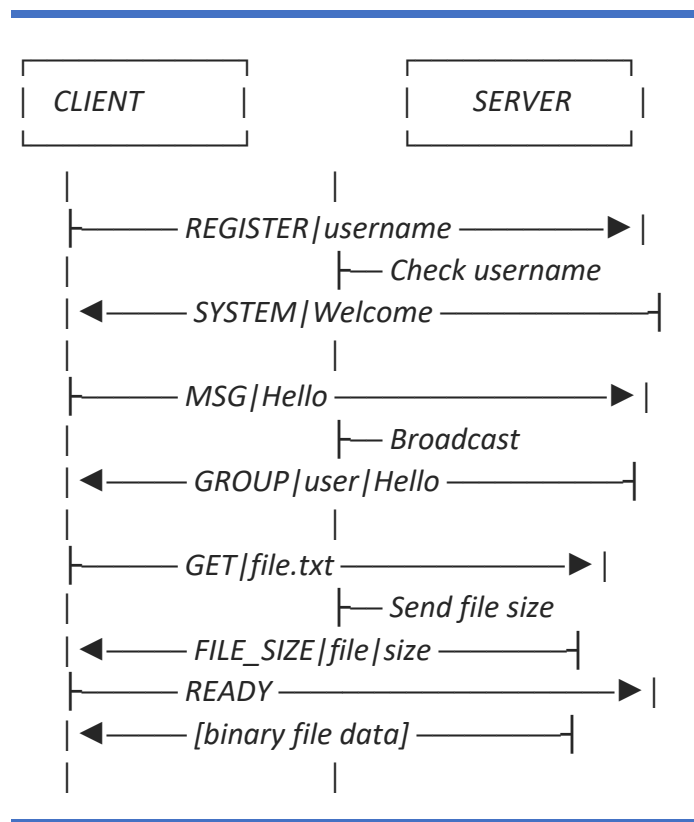
The system consists of two main components:

Server
- Listens on port 8888 (configurable)
- Uses select() for I/O multiplexing
- Maintains a list of connected clients
- Stores files in server_files/ directory



Client
- Connects to server via TCP
- Uses select() for concurrent stdin/socket reading
- Stores files in test_files/ directory
- Color-coded terminal output

2.2 Client-Server Interaction Flow



2.3 Concurrency Model

The server uses select() system call to monitor multiple socket descriptors simultaneously. This single-process, single-thread approach eliminates the need for fork() or pthreads while providing efficient concurrent client handling. All clients share

the same memory space, making user list management and message broadcasting straightforward.

Advantages of `select()` over `fork()`:

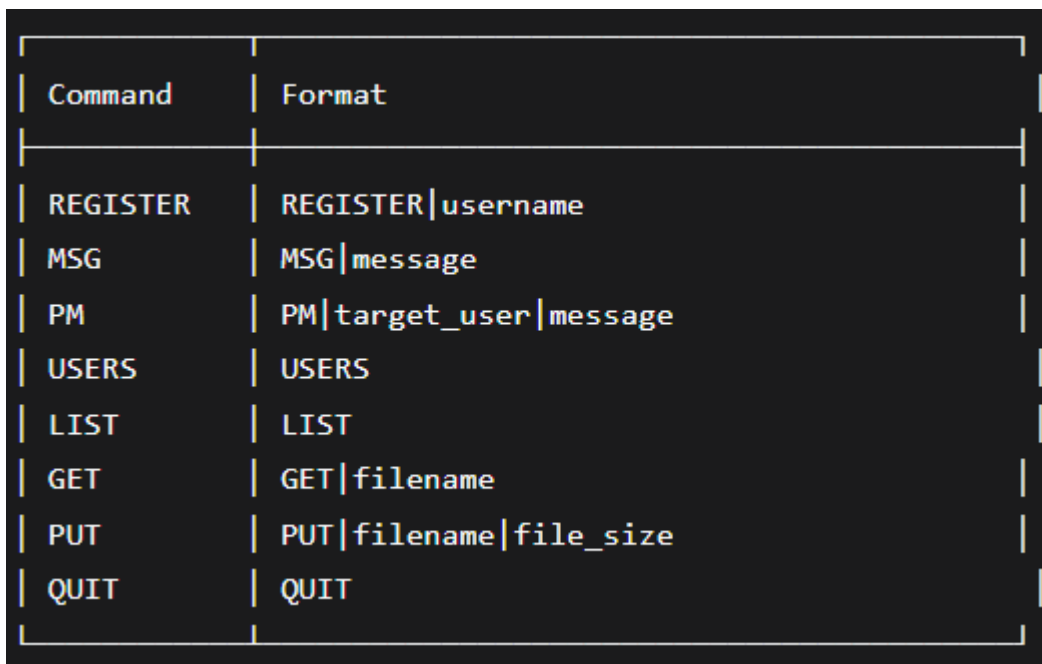
- Shared memory - all clients visible to each other
- No zombie processes to handle
- Simpler code with fewer synchronization issues
- Lower memory overhead per client

3. Protocol Design

All communication uses a text-based protocol with pipe (|) **delimiters**. File transfers use a **binary** format after the initial **handshake**.

3.1 Command Format

Client -> Server:



Command	Format
REGISTER	REGISTER username
MSG	MSG message
PM	PM target_user message
USERS	USERS
LIST	LIST
GET	GET filename
PUT	PUT filename file_size
QUIT	QUIT

Fig 1. Client to server protocol

Server -> Client:

Type	Format
GROUP	GROUP sender message
PRIVATE	PRIVATE sender message
SYSTEM	SYSTEM message
USER_LIST	USER_LIST user1 user2 ...
FILE_LIST	FILE_LIST file:size file2:size ...
ERROR	ERROR message

Fig 2. Server to Client protocol

3.2 File Transfer Protocol

File transfer follows a four-step handshake:

1. Client: GET|filename
2. Server: FILE_SIZE|filename|size
3. Client: READY
4. Server: [binary file data]

This ensures both parties are synchronized before binary data transmission.

4. Implementation Details

4.1 Server Implementation (server.cpp)

The server uses the select system call to handle multiple clients in a single process. The main loop monitors the server socket for new connections and all client sockets for incoming data.

When a new client connects, the server receives a REGISTER command with a username. It checks if the username is already taken by searching the clients vector. If available, the client is added to the vector and a welcome message is sent. If taken, an error is sent and the connection is closed.

For group messages, the server receives an MSG command and broadcasts the message to all connected clients except the sender.

For private messages, the server receives a PM command with a target username. It finds the target client and sends the message only to that client.

For the USERS command, the server builds a list of all authenticated usernames and sends it to the requesting client.

For the LIST command, the server reads the server_files directory and sends back a list of filenames with their sizes.

For the GET command, the server checks if the requested file exists. If it does, it sends the file size, waits for a READY signal from the client, then reads the file in 64KB chunks and sends them over the socket.

For the PUT command, the server checks if a file with the same name already exists. If not, it sends a READY signal, then receives data in 64KB chunks and writes them to a new file in the server_files directory.

When a client disconnects, the server removes it from the clients vector and broadcasts a leave message to all remaining clients.

All socket operations have their return values checked. If any operation fails, the server logs an error and cleans up resources.

The server runs an infinite loop with select() monitoring:

- Server socket (for new connections)
- All client sockets (for incoming data) – each client has a socket of itself

Key functions:

Function	Purpose
handle_new_connection	Accepts and registers new client
handle_client_data	Handles client data
broadcast_message	Sends message to all clients
handle_get_command	Sends file to client

handle_put_command	Receives file from client
--------------------	---------------------------

The server maintains a vector<Client> structure:

```
struct Client {  
    int socket_fd;  
    string username;  
    bool is_authenticated;  
    time_t connect_time;  
};
```

4.2 Client Implementation (client.cpp)

The client uses select to monitor both the server socket for incoming messages and standard input for user keyboard commands. This allows messages to appear while the user is typing.

When started, the client connects to the server and sends a REGISTER command with the entered username. If registration succeeds, it enters the main loop.

For user commands, the client parses the input and sends the appropriate command to the server. For MSG and PM commands, it also displays the message locally. For HELP, it displays a list of commands locally without contacting the server.

For the GET command, the client sends the request to the server. When it receives a FILE_SIZE response, it creates the test_files directory if needed, opens a local file for writing, sends a READY signal, then receives file data in 64KB chunks. A progress bar is displayed using the carriage return character to update the same line.

For the PUT command, the client opens the requested file from the test_files directory, gets its size, and sends a PUT command with the filename and size. After receiving a READY signal from the server, it reads the file in 64KB chunks and sends them over the socket. A progress bar is displayed during the transfer.

For incoming messages from the server, the client checks the message type. GROUP and PRIVATE messages are displayed with appropriate labels. SYSTEM messages are shown as notifications. USER_LIST and FILE_LIST responses are formatted and displayed to the user.

When the user types QUIT, the client sends the command to the server, breaks out of the main loop, and closes the socket.

All file operations use binary mode to preserve exact file contents. All socket and file operations check for errors and display appropriate messages.

The client uses `select()` to monitor:

- Server socket (for incoming messages)
- STDIN (for user keyboard input)

This allows messages to appear even while the user is typing.

File operations:

- PUT: Reads file from `test_files/` directory
- GET: Writes file to `test_files/` directory
- Progress bar updates using carriage return (`\r`)

4.3 Directory Structure

```
project/
├── server      # Server executable
├── client      # Client executable
├── server_files/  # Server's file storage
│   └── [uploaded files]
├── test_files/   # Client's file storage
│   └── [local files]
└── server.cpp, client.cpp
```

4.4 Error Handling

All system calls (`socket`, `connect`, `send`, `recv`) check return values. Errors are displayed with color-coded messages and do not crash the program.

5. Available Commands

Command	Usage & Description
MSG	MSG <message> Broadcast message to all online users
PM	PM <username> <message> Send private message to specific user
USERS	USERS Display list of currently online users
LIST	LIST Show available files on server
GET	GET <filename> Download file from server to test_files/
PUT	PUT <filename> Upload file from test_files/ to server
QUIT	QUIT Disconnect from server and exit
HELP	HELP Display this command reference

6. Compilation & Execution

6.1 Prerequisites

- Linux enviromant
- G++ compiler with C++11 support
- Make utility

6.2 Compilation

\$ make # Compiles both server and client

\$ make clean # Removes compiled binaries

6.3 Running the Server

\$./server # Default port 8888
\$./server 8889 # Custom port

Expected output:

```
=====
FTP & Chat Server - Computer Networks
=====
Server listening on port 8888
File directory: server_files
Waiting for connections...
```

6.4 Running a Client

\$./client # Connects to localhost:8888
\$./client 192.168.1.100 8888 # Custom server

The client will prompt for a username, then display the command prompt (>).

6.5 Testing Multiple Clients

Open separate terminal windows and run ./client in each with different usernames to test group messaging and private messaging.

7. Testing Results

We conducted the testing with 3 users Mahdy, Javad, and Soniya.

7.1 Server Startup

```

mh_mokh7@MahdyBeast:/mnt/c/Users/Notebook/Desktop/programing/Computer Networking/computerNetworking-miniProjects/CA1-FTP
ChatProject$ make clean
Cleaning up...
rm -f server client
Clean complete!
mh_mokh7@MahdyBeast:/mnt/c/Users/Notebook/Desktop/programing/Computer Networking/computerNetworking-miniProjects/CA1-FTP
ChatProject$ make run-server
Compiling server...
g++ -std=c++11 -Wall -Wextra -o server server.cpp -pthread
Server compiled successfully!
Starting server on port 8888...
./server
=====
FTP & Chat Server - Computer Networks
=====
Server listening on port 8888
File directory: server_files
Using select() for multi-client handling
Waiting for connections...
=====
|

```

Fig 3. Server setup

7.2 Client Connection

```

mh_mokh7@MahdyBeast:/mnt/c/Users/Notebook/Desktop/programing/Computer Networking/computerNetworking-miniProjec
ts/CA1-FTPChatProject$ make run-client
Compiling client...
g++ -std=c++11 -Wall -Wextra -o client client.cpp
Client compiled successfully!
Starting client...
./client
=====
FTP & Chat Client - Computer Networks
=====
Enter your username: Mahdy
Connecting to 127.0.0.1:8888... Connected!

=====
Connected as: Mahdy
Type commands below (type HELP for help)
=====

[23:01:49] [System] Welcome to the chat! Type HELP for available commands.
>

```

Fig 4. Client Mahdy connected

7.3 Multiple Clients Online

```

mh_mokh7@MahdyBeast:/mnt/c/Users/Notebook/Desktop/programing/Computer Networking/computerNetworking-miniProjec
ts/CA1-FTPChatProject$ ./client
=====
FTP & Chat Client - Computer Networks
=====
Enter your username: Javad
Connecting to 127.0.0.1:8888... Connected!

=====
Connected as: Javad
Type commands below (type HELP for help)
=====

[23:03:03] [System] Welcome to the chat! Type HELP for available commands.
>

```

Fig 5. Client Javad connected

```

mh_mokh7@MahdyBeast:/mnt/c/Users/Notebook/Desktop/programing/Computer Networking/computerNetworking-miniProjec
ts/CA1-FTPChatProject$ make run-client
Starting client...
./client
=====
FTP & Chat Client - Computer Networks
=====
Enter your username: Soniya
Connecting to 127.0.0.1:8888... Connected!

=====
Connected as: Soniya
Type commands below (type HELP for help)
=====

[23:04:05] [System] Welcome to the chat! Type HELP for available commands.
> USERS

=== Online Users ===
• Mahdy
• Javad
• Soniya
=====
> █

```

Fig 6. Client Soniya, Javad, Mahdy all connected

7.4 Private Message (PM)

```

[23:04:05] [System] Soniya has joined the chat
> PM Soniya hi my friend!
[23:05:36] [You] [Private to Soniya] hi my friend!
> █

```

Fig 7. Mahdy sent a PM to Soniya

```

[23:05:36] [Private] Mahdy: hi my friend!
> █

```

Fig 8. Soniya receives PM from Mahdy

7.5 Group Message (MSG)

```

> MSG im sending message to all the users
[23:07:45] [You] im sending message to all the users
> █

```

Fig 9. Javad sending message to all users

```

[23:07:45] [Group] Javad: im sending message to all the users
> █

```

Fig 10. Mahdy received the group message

```
[23:07:45] [Group] Javad: im sending message to all the users
> █
```

Fig 11. Soniya received the group message

7.6 Online Users List (USERS)

```
> USERS

=== Online Users ===
• Mahdy
• Javad
• Soniya
=====
```

Fig 12. Online users list

7.7 File Listing (LIST)

```
> LIST

=== Available Files ===
• 21.JPG (1032 KB)
=====

> █
```

Fig 13. List of server files

7.8 File Download (GET)

```
> GET 21.JPG
[23:12:03] [File] Downloading 21.JPG (1032 KB)...
Downloading 21.JPG: [=====] 100% (1032/1032 KB)
[23:12:03] [File] Download completed successfully! Saved to: test_files/21.JPG
> █
```

Fig 14. Downloaded the file from the server

7.9 File Upload (PUT)

```
> PUT upload1.txt
[23:12:44] [File] Uploading upload1.txt (0 KB)...
Uploading: 100% (0/0 KB)
[23:12:44] [File] Upload completed successfully!
> █
```

Fig 15. Uploading file to the server

7.10 Client Disconnect (QUIT)

```
> QUIT
[23:14:02] [System] Disconnecting...
>
Disconnected from server. Goodbye!
```

Fig 16. Exiting from the group chat

```
[23:01:49] New connection from 127.0.0.1:45356
[23:01:49] User 'Mahdy' registered. Total clients: 1
[23:03:03] New connection from 127.0.0.1:48960
[23:03:03] User 'Javad' registered. Total clients: 2
[23:04:05] New connection from 127.0.0.1:52252
[23:04:05] User 'Soniya' registered. Total clients: 3
[23:04:13] Sent user list to client. Total users: 3
[23:05:36] User 'Mahdy' sent private message to 'Soniya'
[23:07:45] User 'Javad' sent group message: im sending message to all the users
[23:10:14] Sent user list to client. Total users: 3
[23:11:19] Sent file list to client
[23:12:03] Sending file: 21.JPG (1032 KB)
[23:12:03] File sent successfully: 21.JPG
[23:12:44] Receiving file: upload1.txt (0 KB)
[23:12:44] File uploaded successfully: upload1.txt
[23:14:02] User 'Mahdy' requested quit
[23:14:02] User 'Mahdy' disconnected. Total clients: 2
```

Fig 17. The server's background process

8. Conclusion

The FTP and Group Chat system was successfully implemented with all required features:

- ✓ Multi-client server using select()
- ✓ Group messaging
- ✓ Private messaging
- ✓ Online user listening
- ✓ File listings
- ✓ File download with progress bar
- ✓ Clean disconnect

The system demonstrates practical understanding of:

- TCP socket programming in c++
- Concurrent IO handling using select()
- Binary file transfer over sockets
- Slient-server protocol design
- Terminal UI with color coding

The project successfully meets all assignment requirements and provides a functional FTP + Chat system.